



Extracting Process Instances from User Interaction Logs

Lars, Kornahrens
August-Wilhelm Scheer Institut für
digitale Produkte und Prozesse
gGmbH
lars.kornahrens@aws-institut.de

Sebastian, Kritzler
August-Wilhelm Scheer Institut für
digitale Produkte und Prozesse
gGmbH
sebastian.kritzler@aws-institut.de

Dirk, Dr. Werth
August-Wilhelm Scheer Institut für
digitale Produkte und Prozesse
gGmbH
dirk.werth@aws-institut.de

ABSTRACT

Thoroughly documenting digital business processes in a company is a crucial and necessary, yet cumbersome task. However, having detailed documentation of one's processes in a modelling language like Business Process Model and Notation (BPMN) can prove very useful regarding process optimization or automation, employee training and on- and offboarding. Process and task mining frameworks try to ease the creation of process documentation by automatically generating it based on transaction or user interaction data with the system. These approaches often have the disadvantage of not covering the whole process due to a variety of possible execution paths and their habit of not continuously recording process data. We propose an extension to the task mining tool Desktop Activity Mining (DAM) which allows to capture data continuously over several hours and therefore not miss any important cases that might not occur very often. This approach also limits the influence of human errors when recording process data with certain frameworks for documentation purposes and provide the possibility of an improved degree of automation. We evaluate the approach on real-world data to show its feasibility and application in practice. We used a combination of already existing algorithms and created our own. By classifying 332 unique user interactions, we end up with 76 different equivalence classes. Evaluating the algorithm, we achieved a classification correctness of 70% in two datasets.

CCS CONCEPTS

• Markov processes; • Stochastic processes; • Probability and statistics;

KEYWORDS

Process Mining, Task Mining, Event Log Processing, Pattern Matching

ACM Reference Format:

Lars, Kornahrens, Sebastian, Kritzler, and Dirk, Dr. Werth. 2022. Extracting Process Instances from User Interaction Logs. In *2022 4th International Conference on Advanced Information Science and System (AISS 2022)*, November 25–27, 2022, Sanya, China. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3573834.3574538>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

AISS 2022, November 25–27, 2022, Sanya, China

© 2022 Association for Computing Machinery.

ACM ISBN 978-1-4503-9793-3/22/11...\$15.00

<https://doi.org/10.1145/3573834.3574538>

1 INTRODUCTION

Detailed knowledge and documentation of digital processes that exist in a company is a key to business success. Documenting existing processes, e.g. in the form of process models, achieves process transparency which is necessary for process management, optimization and automation, and educational purposes. Detailed process knowledge, especially knowing how a process is executed in all details, can reveal bottlenecks and inefficiencies that can be tackled by optimizing and changing the process. Thorough process documentation might also uncover automation potentials due to very repetitive and structured tasks in a process [1]. Furthermore, educating employees regarding a new process they should execute proves much easier if detailed documentation of a process with all its exceptions and special cases exists. However, in practice, the existing process documentation in a company is often not detailed enough to provide a thorough understanding of the process or it is out-of-date due to changes in the process. This makes it extremely difficult to rely on existing models for further business decisions. Furthermore, even though the models might be detailed and up-to-date, they might only cover the most common cases of the process and neglect any special scenarios and cases that do not occur on a daily basis.

To overcome the mentioned issues, there are task mining solutions that create process documentation based on recordings of real process executions, the process instances. In this context, the executed work steps along the process are viewed from an operational perspective, i.e. how something was executed. The market offers different Task Mining solutions, providing a varying feature set for the user. After an examination of the Task Mining market, we have decided to use DAM [2] for different reasons. DAM is an in-house developed tool that captures user interactions with the system in a very high level of details. DAM records all events like mouse clicks and keyboard inputs, i.e. user interactions, that happen during the execution of a process, regardless of the used software system. In this way, event-based data on the executed process can be recorded, centrally stored and processed comprehensively and in a high degree of detail on one platform. By capturing several process instances we are able to generate a fine-grained process model that includes all possible process scenarios of an organization. Processes and process instances need to be explicitly recorded and thoroughly executed regarding the start and end points as well as interruptions during the process execution. If some unrelated events are recorded, the correctness of the generated process model cannot be guaranteed anymore.

To enable uninterrupted and accurate recording of event data generated during process execution, we intend to extend DAM with a continuous recording functionality. Our approach is a continuous

recording over several hours which contains several process executions without the need to start and stop the recording process for new process executions. This limits the possibility for human error in the recording process and makes it easier to keep the process documentation up-to-date as the effort to do is reduced. Undoubtedly, this approach brings new challenges. We need a reliable way of identifying the single process executions within the long recording. This includes identifying the start and end points of process executions and detecting noise between executions. The identification of the start and end points of the process execution is also necessary so that the event data collected therein can be assigned to the associated process instance in a time-related sorted manner, also known as a trace. Only after this assignment can the event data in the traces be analysed in a process-oriented manner, so that the process instances are correctly represented in a process model. Our approach consists of two steps. First, we preprocess the recorded log data by grouping the recorded user interactions into event equivalence classes. This reduces the diversity of the log and makes it more robust for the following step. The second step involves the use of Markov chains, which are often used for sequence clustering and classification due to their ability to capture complex chronological dependencies [3]. Here we apply adapted Markov chains and a scoring function based on pattern matching to the reduced log and estimate the start and end points of a process instance. We apply the second step iteratively until no changes in the detection result occur and to increase the probability that all process instances in the log are recognized.

We evaluate our approach on two real-world datasets. Our developed preprocessing technique can reduce the diversity of the user interaction log to less than a quarter of the original diversity. Moreover, the extracted instances are very accurate and deviate less than a total of six user interactions at the start and the end of the instance in more than 90% of the cases, while using three training instances. The accuracy is stable across the evaluated datasets, however, the number of retrieved instances changes immensely due to the usage of only three training instances.

The remainder of the paper is structured as follows. Section 2 covers all required background information and related work. Section 3 describes the preprocessing of the event log to make it suitable for our approach. In Section 4 we define our developed algorithm. Section V contains the evaluation of the approach and Section 6 concludes the paper and opens possible future research questions.

2 BACKGROUND AND RELATED WORK

In the proceedings of creating the algorithm, we searched for similar approaches to our use case. We specifically searched for approaches to Clustering and Mining Patterns in event logs, but found none that are particularly useful for our case. To further give some background to our work, we give a more detailed view on DAM, including some information about technical and operative characteristics.

2.1 Desktop Activity Mining (DAM)

DAM is a task mining tool that can be used to record and analyze digital processes [2]. For each interaction between the user and the software system, DAM generates a screenshot using a recording

software. The interactions include but are not limited to keystrokes or mouse clicks. Furthermore, DAM extracts an identifier for the process instance from the screenshots. This identifier can be retrieved from the Microsoft UI Automation API for desktop applications. If an event occurs in the browser, the identifier consists of the URL and the HTML element including its embedding hierarchy.

The identifiers are an important component for creating traces, since each event must be assigned to its process instance. In addition, it is necessary to sort the events within a trace chronologically in ascending order by means of the time stamp. This means that the executed interactions inside the work steps along an executed process are mapped in fine granularity in the traces. Several traces of a process are stored in an event log. The process model is finally generated from the event log. A process model consists of a set of process instances of a process. The beginning and end of a process instance must be clearly defined. Otherwise, it may happen that the traces overlap, or event data can no longer be assigned to the corresponding process instance through the identifier. These anomalies are also known as noise. For manual handling of the recorder software, the user must therefore ensure that the recorder is started and stopped before and after the process instance. This makes the use of DAM or the recorder very prone to human error, e.g. forgetting to stop the recording. This highlights the need to provide DAM with the functionality to continuously record the event data during the execution of multiple process instances.

2.2 Clustering and Mining Patterns in Event Logs

Clustering log data and finding patterns in log files is a field widely discussed in literature. There are several approaches that make use of pattern mining techniques to structure event logs. Hsu et al. [4] propose an approximate pattern matching algorithm to extract repeating patterns from sequence data by using a sliding window approach and apply their approach to musical data. Their approach generates a huge amount of possible approximate patterns for a sequence as no restrictions regarding pattern length, sub patterns, or other parameters are made. Walicki and Ferreira [5] perform a pattern search using a complete search of the whole search space, i.e. all possible patterns, to detect sequences in unlabeled event logs. For large logs, this approach is not feasible as the number of potential search candidates is too big.

Apart from pattern matching algorithms, also clustering-based techniques occur with a variety of characteristics. Kwon et al. [6] propose an online clustering algorithm of event logs that works based on additional context information of each event. This context is defined based on the use case scenario and also requires the definition of a distance metric to assess the similarity of two events. Makaju et al. [7] perform mining of clusters from the event logs based on the occurrence of special tokens in the logs. Vaarandi [8] describes a data clustering algorithm that is based on the words (textual information) in the event logs. The requirements for this algorithm are the existence of many unique words under the assumption of a strong correlation between the words that do occur frequently. All of the mentioned clustering approaches work on events logs that do not contain user interactions, but rather generic events like ‘Start billing procedure for customer ABC, order XYZ’.

Therefore, we cannot directly apply the proposed approaches to our user interaction log.

Other approaches include a genetic algorithm proposed in [9] by de Medeiros et al. The algorithm is used to find patterns in log files which can then be used to create process models. However, the approach only works on data sets where there is a limited number of different event types, which is not the case in our user interaction log. Another idea is presented by Verbeek et al. [10]. To work with large datasets, they split the dataset and apply established algorithms on the smaller subsets. In the end, the results of the subsets are combined and achieve higher accuracy than processing the dataset as a whole would. Günther et al. [11] propose a global trace segmentation algorithm that groups low-level events in log files in order to find patterns that represent an activity in the context of process mining.

Ferreira and Gillblad [12] present an algorithm based on Markov models to extract process models from unlabeled event logs which do not necessarily have to contain an instance identifier for the log entries. Their approach can deal with interleaving instances and can also be used to extract information for instance identification. One huge drawback of the approach is that it only works for a very limited number of different event classes in the log file.

Several approaches deal with the problem of cleaning logs before using the log for further applications. As van Zelst et al. outline in their literature review in [13], most of the existing approaches try to use event abstraction to achieve a diversity reduction, simplification and generalization of the elements in event logs. The processed log can then be used for further applications like process mining. Another approach that relies on additional information about the underlying process is presented in [14] by Ly et al. They clear logs based on semantic information about the process model. Therefore, their approach needs semantic information about the process that is not available in our event logs.

More recent literature in the field of process mining like [15] and [16] deals with the mining of process models from a set of already separated instances rather than the actual splitting of process instances in a continuous recording of user interactions. If an employer records the execution of a business process one usually ends up with a sequence of user interactions which are not labelled by process instances and therefore the in [17] reviewed state-of-the-art algorithms cannot be directly applied. Hence our approach can be seen as a necessary part for the automatic generation of process models from continuous recordings of user interactions.

All the presented approaches have certain drawbacks which make them unfit particularly for our goal of separating process instances in a log of user interactions. Therefore, we combine elements from the approaches presented in [4] and [12] to create our own algorithm. This will be presented in more detail later in the evaluation.

3 EVENT LOG PREPROCESSING

This section covers an explanation of the used data model and the preprocessing of the recorded user interactions log from DAM. Preprocessing the data is an important step in our approach as it immensely influences the performance of the automated recognition

and separation of multiple process instances in a longer recording of user interactions. Without preprocessing the data, the user interaction log is far too diverse, i.e. contains too many different elements, to be used for any meaningful analysis.

The following definition formally introduces the notion of user interaction logs and their elements.

Definition 1: A user interaction is defined as n-tuple = $\{u_1, u_2, \dots, u_n\}$, with $u_i \in \Omega_i$ and $U \in \Omega$, where Ω_i denotes the space of possible values for the i-th attribute of the user interaction and $\Omega = \times_{i=1}^n \Omega_i$ denotes the space of all possible user interactions. A user interaction log S of length k is defined as a sequence of user interactions denoted with $S = \{s_j\}_{j=1}^k$ for $s_j \in \Omega$. The set of user interactions contained in S is denoted with Ω_S .

The aim of the preprocessing is to highly reduce the diversity of the user interaction log, the number of different elements. This can be realized by defining an appropriate equivalence relation for user interactions and transforming the sequence S to a sequence $\tilde{S} = \{[s_j]\}_{j=1}^k$, where $[s_j]$ denotes the equivalence class that corresponds to s_j . Based on this equivalence relation we can map several different user interactions to one equivalence class and therefore reduce the number of different elements in the log. We denote the resulting quotient space of Ω modulo the equivalence relation as $\tilde{\Omega}$ and define $\tilde{\Omega} := \{[u] : \forall u \in \Omega\}$. Furthermore, we define the transformed user interaction log as $\tilde{U} := \{[u_1], [u_2], \dots, [u_n]\}$. A transformed user interaction log containing exactly one process instance is called a transformed instance.

Since most of the attributes of a user interaction are non-numeric, we need a suitable string similarity metric for equivalence class assignment. We choose the Levenshtein distance [18] because it is a metric and a special case of the easily understandable class of edit distances. Other edit distances for example the longest common subsequence distance (LCS) does not allow substitution as an edit process and therefore lead to a coarser grouping of strings in the following definition. In addition to that, the Hamming distance cannot be used, because it can only compare strings of equal lengths which is most of the time not the case in the following application. The Levenshtein distance fulfils the requirements to be a metric (identity of indiscernible, symmetry, and subadditivity) and as a consequence, the following relation denoted with $\sim$ is an equivalence relation.

Definition 2: Let Ω be a set of user interactions and $U_1, U_2 \in \Omega$. Furthermore, let $p \in R^n$ be a parameter vector, $d_L(U_j, U_k)$ the Levenshtein distance between U_j and U_k , and $I = \{i \in \mathbb{N} : i \leq n\}$ the index set.

We say $U_1 \sim U_2$ if and only if $d_L(U_1^i, U_2^i) \leq p^i \forall i \in I$, where U_j^k is defined as the k-th attribute of U_j .

The parameter vector p allows us to configure the desired similarity of user interaction attributes. If we chose a value of p to be zero, this means that this attribute should be the same in both user interactions. In contrast, if we choose non-zero parameters, the corresponding user interactions will only belong to the same equivalence group if the specific attributes have a Levenshtein distance lower or equal to the specified parameter value. The exact choice of the parameter values is rather difficult and depends on the context as well as on the chosen attributes. For example, more diverse attributes like URLs or Windows Automation paths should

most certainly not be matched based on equality, but on a specified similarity whereas attributes like the active application name are suitable for an exact comparison.

Before we have a look at our developed algorithm, we describe the used data model and the chosen parameters for determining the equivalence classes. From the vast amount of information stored for one user interaction during the recording with the DAM application, we choose the following eight attributes after careful consideration: ActionType, ClickType, KeyboardInputType, Application, URL, XPath, AutomationName and AutomationElement. The possible values for ActionType are denoted with $\Omega_{action} = \{\text{MouseDown}, \text{KeyboardAction}\}$ and analogously ClickType and KeyboardInputType can be defined as $\Omega_{click} = \{\text{LeftClick}, \text{RightClick}\}$ and $\Omega_{keyboard} = \{\text{special}, \text{alphaNumeric}\}$, respectively. In contrast to the mentioned three finite state spaces, there is no limitation or known list of attribute values for the remaining attributes. Application denotes the name of the application a user interaction was performed in (e.g. Google Chrome). URL and XPath are two attributes that are only available if the user interaction happened in a browser. URL is the full URL of the used website if the user interaction was executed in a browser (e.g. <https://www.bbc.com/>) and XPath shows the detailed embedding of the UI element in the HTML structure of the website (e.g. `/html/body/div[1]/div[2]/div[2]/div/ul/li/a`). AutomationName and AutomationElement are extracted from the Microsoft UI Automation (UIA) API and refer to the visual information of the used UI element (e.g. ‘Send’, ‘Choose file’) and the class/type of the UI element (e.g. button, UITextbox, UIProperty), respectively. In contrast to the other extracted attributes, AutomationName and AutomationElement are highly dependent on the quality of the extracted information from the UIA API. For some applications, the extracted information is very detailed which makes identifying UI elements correctly rather easy, whereas for others it is quite sparse and therefore not that useful. Nevertheless, we include the attributes in our data model because they provide valuable information about the involved UI elements in the user interaction. Among the recorded attributes from DAM that we do not use are exact click coordinates, values of keyboard inputs, and screenshots. The click coordinates are too dependent on particular user preferences and hardware settings to be useful for our scenario. Apart from the fact that a user never clicks on the same coordinates even if he uses the same UI element, the screen resolution affects the coordinates just as much as using an application not in full-screen mode and placing it in different positions on the screen. The keyboard input most of the time changes for each execution of the process and can therefore not be compared across multiple executions. The recorded screenshots could provide valuable insights if they are used for a visual comparison of the relevant UI elements. However, this is out of the scope of our approach.

The resulting space of user interactions Ω can be defined as $\Omega = \Omega_{action} \times \Omega_{click} \times \Omega_{keyboard} \times \Omega_{application} \times \Omega_{url} \times \Omega_{xpath} \times \Omega_{autoName} \times \Omega_{autoElement}$. For the parameter vector p we choose $p = (0, 0, 0, 0, 10, 10, 10, 6)$ which means that the first four attributes have to match exactly and diverse attributes like XPath, URL, or the Windows Automation attributes are compared on the basis of a certain threshold. The selection of this parameter vector is based

on an experimental evaluation which is described in more detail in Section 5.

To sum up, the preprocessing step takes a recorded user interaction log and simplifies it by reducing the number of different elements in it. This is achieved by assigning all user interactions to equivalence classes that represent the same event. The transformed log is less diverse than the original log and can therefore be used more easily in the following steps of our approach.

4 INSTANCE RECOGNITION AND SEPARATION

For the recognition and separation of process instances in the transformed log $\tilde{S}$, we use a supervised learning approach. Since our algorithm needs to be easily applicable to new and unknown processes, the required amount of training data must be minimal which usually results in poor performance of common supervised learning strategies. To overcome this problem, we present a completely new approach based on Markov chains and a scoring function based on pattern matching. Our approach only needs little training data in form of training instances that are correctly classified concerning their start and end points. In the remainder of the section, we thoroughly introduce the idea of the involved concepts and explain how the methods can be combined to an iterative learning approach.

4.1 Pattern matching

This section covers our approximate pattern matching approach including a scoring function that calculates the probability that a given subsequence of a user interaction log is a process instance. The goal of pattern matching is to further reduce the diversity of the transformed user interaction log by grouping consecutive user interactions into recurring patterns.

Since exact pattern matching will lead to poor results in noisy sequences, we choose an approximative pattern matching approach as the foundation for our instance separation. Because of its simplicity and the parameterizable algorithm we use the approach presented in [4] and apply it to the transformed user interaction log from the preprocessing step. The algorithm usually finds more approximate patterns, than there are unique elements, i.e. user interactions (or to be specific the event classes they belong to), in the sequence. This is a problem because we want to decrease the diversity of the user interaction log and not increase it. Therefore, we union similar approximate patterns of the same length and furthermore remove patterns of length n if they are contained in patterns of length $n + 1$. We say that two different patterns of the same length are similar to each other if they only differ in a low number of different elements. For patterns of length two they need to be the same and for patterns of a length between three and five, only one differing element is allowed. For patterns with length six to seven, we allow two differing elements. Since we set the maximal pattern length in the presented algorithm to seven, we propose no definition of similarity for longer patterns. The upper bound of seven for the maximal pattern length is chosen because the search for longer patterns does not provide a better performance in our use cases. Even though we try to find as many patterns as possible in the user interaction log, not all user interactions will be matched to a pattern found by the algorithm. Therefore, we assign the remaining user

interactions to a neighboring pattern to avoid a log that contains a mixture of patterns and user interactions. A drawback of this approach is the strong simplification of the user interaction log. Elements that occur less often than a certain frequency are not considered for the pattern matching and are therefore not properly represented in the resulting pattern log.

Using Markov chains on a transformed pattern log is not able to generate accurate predictions about the start and end points of instances since the transformed pattern log is too simplified. Therefore, we use a similarity function that predicts the probability for a subsequence of a user interaction log to be an instance based on common patterns to training instances and the length of the subsequence.

Definition 3: Let T be a set of training instances, $\mu_{length} \in \mathbb{N}$ be their mean length and $p_{common} \in \mathbb{N}$ the number of common patterns in the training instances. Furthermore, let U denote a user interaction log. For a tuple (l, p) , where l is the length of a subsequence S and p is the number of patterns which are included in the training instances as well as in the transformed instance, the function $f_{score} : \mathbb{N} \times \mathbb{N} \rightarrow [0, 1]$, $(l, p) \mapsto \frac{1}{2} \left(1 - \frac{1}{5} \frac{|\mu_{length} - l|}{|\mu_{length} + l|} \right) + \frac{1}{4} \left(1 + \sqrt{\frac{p}{p_{common}}} \right)$ displays how probable it is for S to be an instance. In other words, f_{score} calculates the probability that S is a process instance depending on the length of the subsequence and the common patterns between the training instances and the subsequence. The specific use of this function will be explained in Section 4.3.

Algorithm 1 Creating transition matrices

```

procedure CreateTransitionMatrices( $\tilde{T}$ ,  $\omega$ )
   $k \leftarrow$  Number of different elements in  $\tilde{T}_1 \in \tilde{T}$ 
   $M_{start} \leftarrow 0^{k \times k}$ 
   $M_{end} \leftarrow 0^{k \times k}$ 
  for  $\tilde{T}_i \in \tilde{T}$  do
     $start \leftarrow$  IDs of first  $\omega$  elements of  $\tilde{T}_i$ 
     $end \leftarrow$  IDs of last  $\omega$  elements of  $\tilde{T}_i$ 
    for  $j \leftarrow 1, \text{length}(start)$  do
       $M_{start}[start[j], start[j+1]] += 1$ 
    end for
    for  $l \leftarrow 1, \text{length}(end)$  do
       $M_{end}[end[l], end[l+1]] += 1$ 
    end for
  end for
   $M_{start} \leftarrow M_{start}$  with normalized row sums
   $M_{end} \leftarrow M_{end}$  with normalized row sums
  return  $M_{start}, M_{end}$ 
end procedure

```

4.2 Markov Model

In general, the representation of processes as Markov chains [19] seems intuitive and has been used for example in [12]. However, because our approach uses little training data the transition matrices will be quite sparse which leads to a high error. This phenomenon occurs since the trained Markov chains are not able to deal with newly occurring event sequences in the log. As this will usually be the case in real-world data, simply applying regular Markov chains

on the user interaction log is not possible. A solution to this problem could be a regularization with a small value to avoid zero probabilities. Nevertheless, the resulting probabilities will be extremely low and not suitable for determining whether a subsequence of the transformed log $\tilde{U}$ is a process instance.

Under the assumption that the same process almost always has a similar beginning and a similar ending, training the Markov Chains only on the events included in the beginning and end sequences of a process seems a promising approach.

Algorithm 1 describes the procedure of creating the transition matrices M_{start} and M_{end} which contain the transition probabilities for user interactions for start and end sequences of the training instances. $\tilde{T}$ denotes the set of training instances and ω the length of the subsequences which will be considered for the creation of the matrices.

Once the transition matrix is constructed, the idea is to iterate over subsequences $\{k_i\}_{i=j}^{j+\omega}$ where k_i is defined as the i -th element of the transformed log $\tilde{U}$, for a given window size ω and determine the probability for the subsequence to be a start sequence or an end sequence of the process. If a subsequence receives non-zero probabilities for being both, a start and an end sequence, it will be classified as the one with the higher probability.

The result of applying the Markov chain to the transformed user interaction log is a list of possible start and end sequences with their corresponding probabilities. This information is combined with the information from the pattern matching step and the resulting approach is described in Section 4.3.

The function *clean* looks at windows of size ω and deletes all candidate endpoints and startpoints apart from the ones with the highest probability computed with the transition matrices M_{start} and M_{end} . Therefore, after the use of *clean* in every window of size ω there is at most one candidate startpoint and one candidate endpoint.

4.3 Combined Iterative Approach

This section describes how the presented building blocks are combined to extract process instances from the user interaction log. First, the preprocessing is applied to the raw user interaction log to create a transformed user interaction log with the corresponding event equivalence classes. This reduced log is then passed on to the Markov model and the pattern matching algorithm. The results of both approaches are combined to create the final separation of the user interaction log into separate instances.

Based on the detected start and end sequences from the Markov chains we match suitable pairs of them and define the subsequence from a start to end sequence as a process instance. As the transformed log $\tilde{U}$ will most likely be noisy in real-world data, this approach will not be able to detect a large part of the recorded process instances. Therefore, we combine this approach with detection based on the approximate pattern matching approach.

Noisy data or a process that violates assumptions (e.g. that a process always has similar start and end sequences) can lead to the fact that not all start and end sequences are detected. Therefore, it can be the case that some required start or end sequences are missing and we cannot simply classify an instance based on a detected start sequence and the following end sequence. To overcome this problem, we start at the first classified start sequence that is retrieved

Algorithm 2 PredictInstances

```

procedure predictInstances( $\tilde{S}$ ,  $M_{start}$ ,  $M_{end}$ ,  $\omega$ )
  candidateStartPoints  $\leftarrow$  []
  candidateEndPoints  $\leftarrow$  []
   $n \leftarrow \text{length}(\tilde{S})$ 
   $i \leftarrow 1$ 
  while  $i + \omega \leq n$  do
    startProb  $\leftarrow 1$ 
    endProb  $\leftarrow 1$ 
    for  $j \leftarrow i:i+\omega$ 
      startProb  $\leftarrow \text{startProb} * M_{start}(j,j+1)$ 
      endProb  $\leftarrow \text{endProb} * M_{end}(j,j+1)$ 
      if startProb > endProb
        candidateStartpoints.append( $i$ )
      else
        candidateEndpoints.append( $i$ )
    end for
  end while
  startPoints, candidateEndPoints  $\leftarrow \text{clean}(\text{candidateStartPoints}, \text{candidateEndPoints})$ 
  for startPoint in startPoints
    if there exists a perfect matching endpoint endPoint for startPoint
       $\tilde{T}_{new}.\text{append}(\tilde{S}[\text{startPoint} : \text{endPoint}])$ 
    else if there exist multiple possible endpoints
      possibleEndpoints
        candidateProbabilities  $\leftarrow$  []
        for possibleEndpoint in possibleEndpoints
           $p \leftarrow \text{common patterns of } \tilde{S}[\text{startPoint}:\text{possibleEndpoint}]$ 
          and training instances
           $\mu \leftarrow \text{possibleEndpoint} - \text{startPoint}$ 
          candidateProbabilities.append( $[\text{possibleEndpoint}, f(\mu, p)]$ )
        end for
       $\tilde{T}_{new}.\text{append}(\tilde{S}[\text{startPoint}, \text{endPoint}])$ , where endPoint is
      the possible endPoint with the highest score stored
      in candidateProbabilities
    end if
  end for
  return  $\tilde{T}_{new}$ 
end procedure

```

Algorithm 3 Getting final predictions

```

procedure GetFinalPredictions( $\tilde{S}$ ,  $\tilde{T}$ ,  $\omega$ )
   $M_{start}, M_{end} \leftarrow \text{CreateTransitionMatrices}(\tilde{T}, \omega)$ 
   $\tilde{T}_{new} \leftarrow []$ 
  while  $\text{length}(\tilde{T}) \neq \text{length}(\tilde{T}_{new})$  do
     $\tilde{T}_{new} \leftarrow \text{predictInstances}(\tilde{S}, M_{start}, M_{end}, \omega)$ 
     $M_{start}, M_{end} \leftarrow \text{CreateTransitionMatrices}(\tilde{S}, \tilde{T}_{new})$ 
  end while
  return  $\tilde{T}_{new}$ 
end procedure

```

by the Markov chain approach. From there on we have a look at the occurrence of the next predicted end and the next predicted start sequence. If the ending of the end and the beginning of the start sequence fall together, we can already assume that the subsequence beginning at the first classified start sequence and ending at the next end sequence is an instance. We now proceed to detect the next instance at the beginning of the next classified start instance. Because of violated assumptions or noise it can happen that the classified start and end sequences are not directly next to each other. The handling of this case is a bit more delicate. If there is an end sequence found in between the start of the current start sequence and the next classified start sequence, it is not clear whether the instance is ending at the classified end or the next classified start sequence if we assume no noise in between the recordings. Since this question cannot be answered with the help of Markov chains, we use the score function f from Section 4.1. We evaluate the score of the subsequence from the beginning to the classified end sequence and the score of the subsequence from the beginning to the next classified start sequence. We choose the subsequence with the higher score to be classified as an instance and start again from the end of the newly predicted instance.

Because of these non-trivial cases where we need to combine the Markov predictions with the probabilities of our score function, we can find instances which the Markov approach was not able to detect initially and either only the start sequence of the instance or only the end sequence of the instance was found. The newly found instances can now be added to the training instances and we update our transition matrices with the new training data. Through this approach, we are able to iteratively predict the splitting points of the instances. The described iterative approach is said to converge if the number of found instances in the user interaction log does not change in the next iteration. The exact algorithm is depicted in Algorithm 2. $\tilde{S}$ denotes the preprocessed subsequence on which we perform our instance extraction and $\tilde{T}$ refers to the set of training instances.

5 EVALUATION

For the evaluation of our approach, we use two real-world datasets. The two Datasets A and B have both been obtained using DAM in different scenarios. Dataset A was recorded by an employee working as an assistant, performing a simple HR related task. Dataset B was obtained from an employee in sales, performing a purchase order.

The executed Process based on the gathered Datasets is illustrated using BPMN in Figure 1. Both Datasets are gathered from standardized Processes, one having minor deviations in a few process instances. Which Datasets to choose had certain criteria to it. It was important for the used Datasets, that the processes are executed in chronological order and one after the other. It can occur that a user is parallelly executing the same Process in several instances. Detecting one process in several instances parallelly, let alone different processes is currently not possible with DAM. Alternative public Datasets did not prove to be suitable for our use case. Dataset A contains 25 process instances of the same business process which has a mean length of 43 with a variance of 50 user interactions. In total, it contains 1087 user interactions of which 143

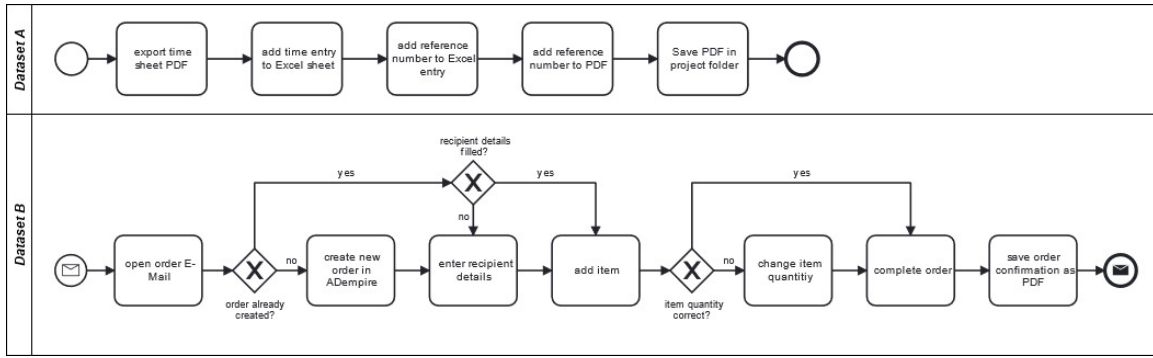


Figure 1: In dataset A and B contained processes illustrated in BPMN

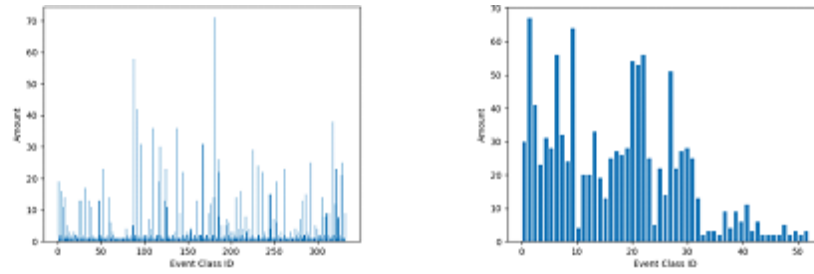


Figure 2: Histogram of user interactions (left) and event classes (right) in dataset A

are unique. Uniqueness, in this case, refers to events that have the same properties regarding the values we chose for evaluating event equivalence classes (see Section 3). These unique events are further clustered into event equivalence classes using our preprocessing approach. Similar to dataset A, dataset B also contains 25 process instances of the same business process with a mean length of 61 and a variance of 100. The total number of user interactions is 1594. With 332 unique user interactions, there are almost twice as many as in dataset A.

Before we discuss the performance of our algorithm presented in Section 4, we have a look at the results from our preprocessing step. Figure 2 display the diversity reduction from the user interaction log to the transformed log in dataset A. We can see a reduction from 143 unique user interactions to only 53 different equivalence classes.

For dataset B we can see an even bigger contrast between the histogram for the unique elements in the original user interaction log and the transformed log with event classes in Figure 3. Instead of 332 unique user interactions we end up with 76 different equivalence classes.

Of course, the reduction of the diversity in the user interaction log, i.e. the number of resulting event equivalence classes, depends on the parameter vector p . The higher we set the values in the parameter vector p the more we will reduce the diversity of the user interaction sequence. The choice of p described in Section 3 is a good compromise between the creating of meaningful event equivalence classes for user interactions and reducing diversity. Since there is no reference grouping of user interactions, we cannot accurately define

what a meaningful equivalence class is nor automatically measure the quality of the grouping. Nevertheless, it is possible to get at least a visual impression of the grouped user interactions. This can be done by looking at the screenshots of all user interactions that are put in the same event equivalence class and assessing their similarity. Like this, we can adjust the entries of p such that the desired level of similarity is achieved. For our approach, we end up with the parameter choice $p = (0, 0, 0, 0, 10, 10, 10, 6)$.

Next, we look at the performance of the instance separation. All values are obtained by using five training instances and the prediction of the other 20 instances. The whole approach is repeated 20 times with a randomly chosen set of the five training instances to get an average performance overview without the careful choice of training. For the approximate pattern matching algorithm from [4] we choose the minimum frequency for the pattern definition in such a way that subsequences have to appear at least four times to be recognized as patterns. Furthermore, we set the approximation degree to one, because otherwise the extracted patterns are too inaccurate to make reasonable predictions with them. As evaluation metrics we choose the relative error in user interactions, the classification correctness, and the percentage of recognized instances. The relative error is the deviation of the predicted start and end points of the process instances to the actual start and end points divided by the number of instances to predict. Furthermore, we say that a process instance is correctly classified if the predicted start and end points do not vary more than three user interactions from the actual start and not more than three user interactions from the actual endpoint. We can make this assumption and accept a small

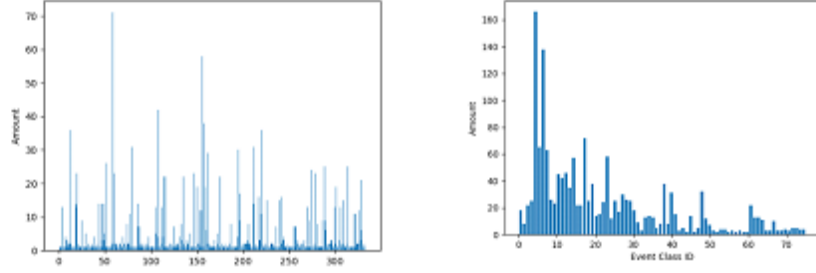


Figure 3: Histogram of user interactions (left) and event classes (right) in dataset B

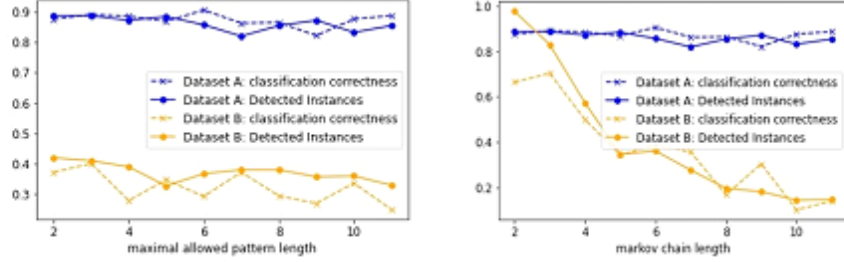


Figure 4: Performance based on the chosen pattern length

error rate of a few user interactions because a deviation that is less than the chosen low threshold is on the level of user actions hardly to distinguish based on the provided screenshot data. Therefore, the notion of correctly classified process instances is, besides the relative error, an interesting performance measure. Apart from that, it is also important to know how many instances the algorithm extracts.

The choice of the remaining parameters for the maximal allowed pattern length and the window size of the used Markov chains are presented in the left chart of Figure 4. For an initial guess of the window size we chose $\omega_{markov} = 5$ and tested the performance of our algorithm based on the chosen maximal allowed pattern size in a range from two to eleven. Values higher than eleven did not have to be evaluated since in both datasets no approximate pattern of length higher than eleven has been found. The percentage of correctly classified and predicted instances strongly differs between dataset A and dataset B, but the changes in those values for a differing maximal allowed pattern length is similar. Further analysis shows a weak negative correlation with pearson correlation coefficient $\rho \approx -0.3$ between the maximal allowed pattern length and the classification correctness in dataset A. In contrast to that Dataset B shows a strong correlation here with $\rho \approx -0.6$. A similar behaviour can be seen for the correlation of maximal allowed pattern length and detected instances.

As a pattern length of three seems to perform best on both datasets, we fixed the maximal allowed pattern length at three and analysed the performance of our algorithm in dependence of the chosen length for the Markov chains which can be seen in the right chart of Figure 4. Whereas for dataset A the length of the Markov chains

is strongly negative correlated to the classification correctness with $\rho \approx -0.91$ an analysis for dataset B only shows a weak correlation of $\rho \approx -0.3$. The behaviour can also be seen for the percentage of detected instances and chosen markov chain length.

The contrasting performance behaviour of dataset A and dataset B can easily be explained. Dataset A contains instances of a very linear process where every instance is quite similar to each other. Because of the similarity in the instances a longer Markov chain will not necessarily negatively affect the prediction behaviour because the training instances cover the behaviour of the instances to predict very well. Dataset B contains instances of a process which has more than one possibility to be executed and where deviations in the process execution can occur shortly after the beginning or shortly before the ending of the process instance, see Figure 1. Throughout our prediction process we only used five training instances which are not able to cover the different possibilities of execution. The choice of using longer Markov chains results therefore in fewer found candidates of start and endpoints and therefore less instances are detected in the log.

Table 1 shows the performance of the presented approach on the two datasets A and B for the maximal allowed pattern length of three and a Markov chain length of three, since this parameter choice seemed to deliver the best trade-off between accuracy and found instances on both datasets. The results show that the mean error is low in both datasets which leads to high overall classification correctness. The biggest difference in performance between the two datasets can be seen in the percentage of recognized instances. One reason for this is the more diverse log in dataset B

Table 1: Performance on dataset A vs. B

Dataset A vs. B	Relative Error (in user interactions)	Classification Correctness (%)	Detected Instances (%)
Mean	1.4 vs. 1.25	0.7 vs. 0.89	0.83 vs. 0.89
Std. dev.	0.7 vs. 0.28	0.19 vs. 0.06	0.19 vs. 0.06
Min.	0.6 vs. 0.7	0.28 vs. 0.88	0.5 vs. 0.85
Max.	3.71 vs. 1.7	1 vs. 1	1.15 vs. 1

due to a less standardized process compared to dataset A. Furthermore, the low number of used training instances directly lowers the number of detected processes instances in dataset B because five training instances cannot fully represent the diverse process starts and endings in the dataset. Therefore, the Markov chains are not able to learn the start and end behaviour of the business process and thus cannot recognize all the instances correctly. In order to overcome this problem, one has to either carefully choose the training instances in such a way that they cover the majority of different process executions or one has to simply use more training instances in order to achieve better coverage of diversity with a higher probability. However, generally choosing a higher number of training instances limits the exact benefit of the approach in that it does not need many training instances.

6 CONCLUSION

In this paper, we present an approach to extract process executions from one long recording of user interactions. This can for example be used for creating a process documentation and therefore process transparency. While in literature only higher-level logs are considered, proposed algorithms for trace segmentation and instance clustering are not designed to handle fine-grained user interaction logs. Therefore, we developed a suitable approach that can separate instances in a continuous recording of user interactions.

Our approach is based on a mixture of an approximate pattern matching approach and Markov chains which allows us to overcome many challenges from previously suggested algorithms. This includes for example the issue of many diverse elements. The achieved classification correctness in both datasets is above 70% which makes our approach especially interesting for semi-automated solutions. A meaningful comparison between different algorithms is hardly possible since contributions in that domain focus on higher-level logs which contain a few different elements compared to the diverse user interaction logs used in our approach.

7 DISCUSSION

This work carries some Limitations to it, which will be addressed as well as giving an outlook into future work. We present an approach that is capable detecting multiple occurrences of a single process in a long recording. However, it is not possible to detect the occurrence of different processes in one log. As our presented approach can be adapted to being able to recognize multiple different business processes in the user interaction log. This is an interesting direction for future work and should be further developed. Furthermore, the amount of noise that the algorithm can handle is limited. The approach is limited in its capability to remove noise between process instances and is currently not able to detect noise within

single process instances. The automatic noise reduction in process instances is quite complicated since the differentiation between newly occurring event sequences that represent noise are hardly distinguishable from newly occurring event sequences representing a new possible execution of tasks in the process, especially if the set of training instances is small which will usually be the case in real-world scenarios. Another limitation of our current approach is the strong dependency on the attributes based on the Microsoft UI Automation API. Since the quality of information that can be retrieved from the API varies for different applications, it may be necessary in some cases to make a different choice for the parameter vector p based on the contained information in those attributes or to manually exclude the affected attributes for certain applications.

REFERENCES

- [1] A. Promitzer, S. Torabi-Goudarzi and D. Werth, "Objectively Assessing the Suitability of Digital Processes for Robotic Process Automation," in 17. Internationale Tagung Wirtschaftsinformatik, 2022.
- [2] C. Linn, P. Zimmermann and D. Werth, "Desktop activity mining-a new level of detail in mining business processes," in Workshops der INFORMATIK 2018- Architekturen, Prozesse, Sicherheit und Nachhaltigkeit, 2018.
- [3] R. Chen *et al.*, "Dynamic order Markov model for categorical sequence clustering," in Journal of Big Data 8, 2021
- [4] J. L. Hsu, A. L. Chen and H. C. Chen, "Finding Approximate Repeating Patterns from Sequence Data," in ISMIR, 2004.
- [5] M. Walicki and D. R. Ferreira, "Sequence Partitioning for Process Mining with Unlabeled Event Logs," Data & Knowledge Engineering, vol. 70, no. 10, pp. 821-841, 2011.
- [6] Y. Kwon, W. Y. Lee, M. Balazinska and G. Xu, "Clustering events on streams using complex context information," in IEEE International Conference on Data Mining Workshops, 2008.
- [7] A. A. Makanju, A. N. Zincir-Heywood and E. E. Milios, "Clustering event logs using iterative partitioning," in ACM SIGKDD international conference on Knowledge discovery and data mining, 2009.
- [8] R. Vaarandi, "A Data Clustering Algorithm for Mining Patterns From Event Logs," in IEEE Workshop on IP Operations & Management, 2003.
- [9] A. K. A. de Medeiros, A. J. Weijters and W. M. van der Aalst, "Genetic process mining: an experimental evaluation," Data mining and knowledge discovery, vol. 14, no. 2, pp. 245-304, 2007.
- [10] H. M. W. Verbeek and W. M. van der Aalst, "Decomposed Process Mining: The ILP Case," in International conference on business process management, 2014.
- [11] C. W. Günther, A. Rozinat and W. M. Van Der Aalst, "Activity mining by global trace segmentation," in International Conference on Business Process Management, 2009.
- [12] D. R. Ferreira and D. Gillblad, "Discovering process models from unlabelled event logs," in International Conference on Business Process Management, 2009.
- [13] S. J. van Zelst, F. d. L. M. Mannhardt and A. Koschmider, "Event abstraction in process mining: literature review and taxonomy," Granular Computing, vol. 6, no. 3, 2021.
- [14] L. T. Ly, C. Indiono, J. Mangler and S. Rinderle-Ma, "Data transformation and semantic log purging for process mining," in International Conference on Advanced Information Systems Engineering, 2012.
- [15] A. Augusto, C. Raffaele, D. Marlon and M. La Rosa, "Split Miner: Discovering Accurate and Simple Business Process Models from Event Logs," in Knowledge and Information Systems, 2019.
- [16] S. Goedertier, J. De Weerd D. Martens, J. Vanthienen, B. Baesens, "Process discovery in event logs: An application in the telecom industry," in Applied Soft Computing, 2011.
- [17] A. Augusto. *et al.*, "Automated Discovery of Process Models from Event Logs: Review and Benchmark," in IEEE Transactions on Knowledge and Data Engineering,

- 2019.
- [18] V.I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," Soviet physics doklady, vol. 10, no. 8, pp. 707-710, 1966.
- [19] P. Brémaud, "Discrete-Time Markov Chains," in Markov Chains, Springer, 2020, pp. 63-109.